

```

1: // Circular Queue implementation in C using Array
2: #include <stdio.h>
3: #define SIZE 5
4: int queue[SIZE];
5: int front = -1, rear = -1;
6: // Check if the queue is full
7: // Suppose there are 5 elements where front is at 0th location and rear
   is at 4th location. In this case,
8: //(front == 0 && rear == SIZE - 1) condition will be satisfied.
9: //if we now delete the element, front will point to 1st location. if we
   insert an element, it will be
10: //inserted at 0th location using the formula (rear=(rear+1)%SIZE then
   (front == rear + 1) condition will
11: //be satisfied.
12: int isFull()
13: {
14:     if ((front == rear + 1) || (front == 0 && rear == SIZE - 1))
15:         return 1;
16:
17:     return 0;
18: }
19:
20: // Check if the queue is empty
21: int isEmpty()
22: {
23:     if (front == -1) return 1;
24:     return 0;
25: }
26:
27: // Adding an element
28: void enqueue(int element)
29: {
30:     if (isFull())
31:         printf("\n Queue is full!! \n");
32:     else
33:     {
34:         if (front == -1) front = 0;
35:         rear = (rear + 1) % SIZE; // Most important
36:         queue[rear] = element;
37:         printf("\n Inserted -> %d", element);
38:     }
39: }
40:
41: // Removing an element
42: int dequeue()
43: {
44:     int element;
45:     if (isEmpty())
46:     {

```

```

47:     printf("\n Queue is empty !! \n");
48:     return (-1);
49: }
50: else
51: {
52:     element = queue[front];
53:     if (front == rear) // Q has only one element, so we reset the
queue after dequeing it.
54:     {
55:         front = -1;
56:         rear = -1;
57:     }
58:     else
59:     {
60:         front = (front + 1) % SIZE; // Most important
61:     }
62:     printf("\n Deleted element -> %d \n", element);
63:     return (element);
64: }
65: }
66:
67: // Display the queue
68: void display()
69: {
70:     int i;
71:     if (isEmpty())
72:         printf(" \n Empty Queue\n");
73:     else
74:     {
75:         printf("\n Front -> %d ", front);
76:         printf("\n Items -> ");
77:         for (i = front; i != rear; i = (i + 1) % SIZE) //Most important
78:         {
79:             printf("%d ", queue[i]);
80:         }
81:         printf("%d ", queue[i]);
82:         printf("\n Rear -> %d \n", rear);
83:     }
84: }
85:
86: int main()
87: {
88:     // Fails because front = -1
89:     deQueue();
90:
91:     enqueue(10);
92:     enqueue(20);
93:     enqueue(30);
94:     enqueue(40);

```

```

95:   enqueue(50);
96:
97:   // Fails to enqueue because front == 0 && rear == SIZE - 1
98:   enqueue(60);
99:
100:  display();
101:  dequeue();
102:
103:  display();
104:
105:  enqueue(70);
106:  display();
107:
108:  // Fails to enqueue because front == rear + 1
109:  enqueue(80);
110:
111:  return 0;
112: }
113:
114:
115: // Circular Queue Using Linked List
116: #include <stdio.h>
117: #include <stdlib.h>
118:
119: typedef struct Node
120: {
121:     int data;
122:     struct Node* next;
123: } Node;
124:
125: Node* front = NULL;
126: Node* rear = NULL;
127:
128: void enqueue(int value)
129: {
130:     Node* newNode = (Node*)malloc(sizeof(Node));
131:     newNode->data = value;
132:     if (front == NULL)
133:     {
134:         front = newNode;
135:         rear = newNode;
136:         rear->next = front; //Most Important Line
137:     }
138:     In a circular queue implemented using a linked list, the line rear->next = front; is used to
139:     maintain the circular structure of the queue.
140:     In a standard linked list, each node has a next pointer that points to
141:     the next node in the

```

```

142: first node (the front node), creating a circular linkage.
143:
144: When a new node is inserted and becomes the new rear node, we update
    the next pointer of the
145: previous rear node to point to the new rear node. This ensures that the
    old rear node is
146: linked to the new rear node, preserving the circular structure.
147:
148: So, the line rear->next = front; sets the next pointer of the rear node
    to point back to the
149: front node. This way, when we traverse the circular queue, we can loop
    back from the last
150: node to the first node, forming a circular path.
151:
152: Without this line, the circular linkage would be broken, and we would
    lose the circular
153: property of the queue.
154: As a result, traversing the queue would not be possible, and it would
    function as a regular
155: linked list rather than a circular queue.
156: */
157:     }
158:     else
159:     {
160:         rear->next = newNode;
161:         rear = newNode;
162:         rear->next = front;
163:     }
164:     printf("Enqueued element: %d\n", value);
165: }
166:
167: void dequeue()
168: {
169:     if (front == NULL)
170:     {
171:         printf("Queue is empty. Cannot dequeue element.\n");
172:         return;
173:     }
174:     int dequeuedValue = front->data;
175:     Node* temp = front;
176:     if (front == rear)
177:     {
178:         front = NULL;
179:         rear = NULL;
180:     }
181:     else
182:     {
183:         front = front->next;
184:         rear->next = front;

```

```

185:     }
186:     free(temp);
187:     printf("Dequeued element: %d\n", dequeuedValue);
188: }
189:
190: void display()
191: {
192:     if (front == NULL)
193:     {
194:         printf("Queue is empty.\n");
195:         return;
196:     }
197:     printf("Queue elements: ");
198:     Node* current = front;
199:     /*
200:      The do-while loop is used to ensure that the loop body is executed
201:      at least once.
202:      This is important because the circular queue can have one or more
203:      elements.
204:      */
205:     do
206:     {
207:         printf("%d ", current->data);
208:         current = current->next;
209:     } while (current != front);
210:     printf("\n");
211: }
212:
213: int main()
214: {
215:     enqueue(10);
216:     enqueue(20);
217:     enqueue(30);
218:
219:     display();
220:
221:     dequeue();
222:     dequeue();
223:
224:     display();
225:
226:     return 0;
227: }

```